

LAB 1

Introduction to C++

From C Structs to C++ Classes

Learning Objectives

By the end of this lab, you should be able to:

- Write a basic C++ program with `#include <iostream>` and using namespace `std`;
- Use `cin` and `cout` for input and output instead of `scanf` and `printf`.
- Use the `string` type instead of char arrays for text data.
- Read a full line of text (including spaces) using `getline(cin, ...)`.
- Recognize the difference between a C struct and a C++ class.
- Explain why class members are private by default — and use the public access specifier when you need them exposed.
- Take the first conceptual step toward object-oriented programming (OOP).

1. Welcome to C++

If you already know C, you know about 80% of C++. C++ keeps everything C can do (you can still use `printf`, `scanf`, and structs) and adds a powerful new toolkit on top — streams (`cin` / `cout`), a real string type, namespaces, and most importantly classes.

In this lab, we will write our first few C++ programs, get comfortable with C++ I/O, and then take the very first step toward object-oriented programming by converting a C-style struct into a C++ class.

Three things every C++ file starts with

`#include <iostream>` — gives you `cin`, `cout`, `endl`.

`#include <string>` — gives you the `string` type (often pulled in automatically by `<iostream>`, but include it explicitly to be safe).

`using namespace std;` — lets you write `cout` instead of `std::cout` every time.

2. Code 1 — Your First C++ Input and Output

Goal: read an ID, a name, and a CGPA from the keyboard, then read a full name (with spaces) using `getline`. This single example introduces almost everything you need for day-to-day C++ I/O.

The Code

```
#include <iostream>
using namespace std;

int main() {
    int id;
    float cgpa;
    string name;

    cout << "Enter your ID, first name and CGPA: ";
    cin >> id >> name >> cgpa;    // name must NOT contain spaces here
    cout << id << " " << name << " " << cgpa << endl;

    // Taking more than one word as input
    cin.ignore();                // clean leftover newline before getline
    cout << "Enter your full name: ";
    string full_name;
    getline(cin, full_name);
    cout << full_name << endl;

    return 0;
}
```

Code Breakdown

Code	What it does
#include <iostream>	← Brings in C++'s input/output stream library — required for cin and cout.
using namespace std;	← Lets us write cout instead of std::cout.
int id; float cgpa; string name;	← Variable declarations. Notice string is a real type — no need for a char array.
cout << "...";	← Print to the screen. The << operator can chain: cout << a << b << c;.
cin >> id >> name >> cgpa;	← Read input separated by whitespace. The name field stops at the first space — that's a problem if names have spaces.
cin.ignore();	← Removes the leftover newline (\n) that >> left behind. Without this, getline would read an empty line.
getline(cin, full_name);	← Reads an ENTIRE line — including spaces — into full_name. The gold standard for reading text in C++.

Sample Run

► Output

```
Enter your ID, first name and CGPA: 101 Rahim 3.85
101 Rahim 3.85
Enter your full name: Mohammad Abdur Rahim
Mohammad Abdur Rahim
```

⚠ Always `cin.ignore()` between `>>` and `getline`

The `>>` operator leaves the newline character (the Enter key) sitting in the input buffer. The next `getline` sees that newline first and treats it as an empty line — so your prompt seems to skip itself. `cin.ignore()` throws that leftover newline away. Make this a reflex.



C-style I/O still works

If you import `<cstdio>`, you can still use *printf* and *scanf* — they were not removed in C++. But *cin* / *cout* are type-safe (they figure out int vs string vs float for you), so prefer them in new C++ code.

3. From C struct to C++ class

In C, when you want to group several related variables together, you use a struct. A Student struct might bundle a name, an id, and a cgpa. The struct is a container for data — but the functions that work on that data live OUTSIDE the struct.

C++ keeps struct AND adds a more powerful sibling: class. A class lets you bundle data AND the functions that operate on it into ONE unit, and it lets you control who is allowed to read or change that data.

The big idea — bundling data with behavior

In C, you write functions that take a struct as a parameter: `printStudent(s1)`. In C++, the function lives INSIDE the class, and you call it on the object: `s1.printStudent()`. This small syntactic change reflects a much bigger shift in how you think about programs.

4. Code 2 — The C Way (struct)

Goal: see how a C-style program organizes a Student record. The data is in the struct; the function that prints it is separate.

The Code

```
#include <stdio.h>
#include <string.h>

struct Student {
    char name[50];
    int id;
    float cgpa;
};

void printStudent(struct Student s) {
    printf("Name: %s\n", s.name);
    printf("ID: %d\n", s.id);
    printf("CGPA: %.2f\n", s.cgpa);
}

int main() {
    struct Student s1;
    strcpy(s1.name, "Rahim");    // can't do s1.name = "Rahim" with char arrays
    s1.id = 101;
    s1.cgpa = 3.75f;
    printStudent(s1);
    return 0;
}
```

What to Notice

- **struct Student just stores data** — it has no functions of its own.

- **printStudent** lives **OUTSIDE** the **struct** and takes the struct as a parameter.
- **Anyone can modify s1.id, s1.cgpa, etc.** from anywhere — there is no protection at all.
- **Strings are clunky** — you need char arrays, *strcpy* to assign, and a fixed maximum length.

Expected Output

► Output

```
Name: Rahim  
ID: 101  
CGPA: 3.75
```

5. Code 3 — The Same Thing as a C++ class (and Why It Fails!)

Now let's translate the same idea into a C++ class. This looks reasonable — but it WILL NOT compile. Read through it first, then we'll see what's wrong.

The Code

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    string name;
    int id;
    float cgpa;
    void printStudent() {
        cout << "Name: " << name << "\n";
        cout << "ID: " << id << "\n";
        cout << "CGPA: " << cgpa << "\n";
    }
};

int main() {
    Student s1;
    s1.name = "Rahim";           // <-- ERROR
    s1.id = 101;                 // <-- ERROR
    s1.cgpa = 3.50;              // <-- ERROR
    s1.printStudent();           // <-- ERROR
    return 0;
}
```

Why It Fails

Every member of a C++ class is PRIVATE by default. private means "only the class itself can touch this." So from inside main, we have no permission to read or write name, id, cgpa, or even to call printStudent. The compiler stops us cold.

Compiler Error

```
error: 'std::string Student::name' is private within this context
error: 'int Student::id' is private within this context
error: 'float Student::cgpa' is private within this context
error: 'void Student::printStudent()' is private within this context
```

This is a feature, not a bug

C++ defaults to private on purpose. The language is gently pushing you to think about WHO should be allowed to touch each piece of data. By default, the answer is "only the class." If you want outside code to access something, you must say so explicitly. This is the foundation of encapsulation, which you'll explore deeply in Lab 2.

 struct vs class — the one tiny difference

A C++ *struct* and a C++ *class* are almost identical. The ONLY built-in difference is the default access level: *struct* members are **public** by default, *class* members are **private** by default. Everything else is the same. By convention, we use *class* for objects with behavior, and *struct* for plain data bundles.

6. Code 4 — Fixing It with public

Goal: make Code 3 compile by exposing the members. We add the public: access specifier so that main can reach the data and call the function.

The Code

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:                                // <-- everything below is accessible from outside
    string name;
    int id;
    float cgpa;
    void printStudent() {
        cout << "Name: " << name << "\n";
        cout << "ID: " << id << "\n";
        cout << "CGPA: " << cgpa << "\n";
    }
};

int main() {
    Student s1;
    s1.name = "Rahim";
    s1.id = 101;
    s1.cgpa = 3.50;
    s1.printStudent();

    Student s2;
    cout << "Enter name, ID, and CGPA for s2: ";
    getline(cin, s2.name);
    cin >> s2.id >> s2.cgpa;
    s2.printStudent();

    return 0;
}
```

Code Breakdown

Code	What it does
class Student {	← Start defining a class. By default, everything inside is private.
public:	← Access specifier — everything BELOW this line is accessible from outside the class.
string name; int id; float cgpa;	← Data members. Now they are public, so main can read and write them.

Code	What it does
<code>void printStudent() { ... }</code>	← Member function — a function that belongs to the class. Inside it, name, id, cgpa are visible WITHOUT writing <code>s1.</code> or <code>s2.</code>
<code>Student s1;</code>	← Create an object of type Student.
<code>s1.printStudent();</code>	← Call the member function ON the object using the dot operator.
<code>getline(cin, s2.name);</code>	← Read a full line (with spaces!) directly into the object's name field.

Sample Run

► Output

```
Name: Rahim
ID: 101
CGPA: 3.5
Enter name, ID, and CGPA for s2: Karim Hossain
201 3.90
Name: Karim Hossain
ID: 201
CGPA: 3.9
```

⚠ Making everything public is a step backward

Code 4 works, but it has thrown away the protection that C++ tried to give us. Anyone can now do `s1.cgpa = -100;` — which makes no sense. **Lab 2** will show the proper fix: keep the data *private* and expose it through controlled *setter* and *getter* functions that can validate the input. That is real OOP.

7. Quick Comparison — C struct vs C++ class

Here is the whole lab in one table:

Aspect	C struct	C++ class
Stores data	Yes	Yes
Stores functions (methods)	No — functions live outside	Yes — functions live inside
Default access	Public (open to all)	Private (closed by default)
String handling	char[] + strcpy	string type (assignment with =)
I/O style	printf / scanf	cout / cin / getline
Encapsulation support	None — every field is exposed	Built-in via private / protected / public
Best for	Plain data bundles	Objects with behavior

8. Summary

Five things to take away from this lab:

- **C++ keeps everything C had** and adds streams (cin/cout), a real string type, and classes.
- **Use cin.ignore() between >> and getline** to throw away the leftover newline. Otherwise getline reads an empty line.
- **A C++ class bundles data AND functions together** in one unit.
- **Class members are private by default.** Use public: to expose them. This is the one and only difference between a struct and a class in C++.
- **Making everything public works but is not the right approach.** Lab 2 will introduce setters and getters — the proper way to expose class data with safety checks.

9. Practice Tasks

Task 1 — Warm-up: Personal Profile

Write a program that does the following:

- Asks the user for their first name (single word) using cin >>.
- Asks for their age (an int) and height in metres (a float) using cin >>.
- Then asks for their full address (a single line, possibly with spaces) using getline.
- Prints back all four values in a neatly formatted block.

 Hint for Task 1

After reading the age and height with `cin >>` you must call `cin.ignore();` BEFORE the `getline` for the address. Otherwise `getline` will read the leftover newline as an empty line and skip the address prompt entirely.

Task 2 — Book Record (from the original lab)

Write a program to manage a book record.

Data Members

- title (string)
- author (string)
- price (double)

Required Tasks

- Define a class `Book` containing the three data members above. Use `public:` for now (we will fix this in Lab 2).
- Add a member function `inputBook()` that prompts the user and reads in all three values.
- Add a member function `displayBook()` that prints all three values in a clean format.
- Add a member function `applyDiscount(double percent)` that reduces price by the given percentage.
- In `main`, create one `Book` object, call `inputBook()`, display it, apply a 10% discount, and display it again to show the updated price.

 Sample interaction

Enter title: The C++ Programming Language

Enter author: Bjarne Stroustrup

Enter price: 1200

--- Book Details ---

Title: The C++ Programming Language

Author: Bjarne Stroustrup

Price: 1200

After 10% discount:

Price: 1080

 Hints for Task 2

Use `getline(cin, title)` for title and author since both can contain spaces.

For the discount: $price = price - (price * percent / 100);$

Remember the `cin.ignore()` trick between reading a number and reading a string with `getline`.

Optional challenge

Try to write `applyDiscount` so that a negative discount or a discount over 100% is rejected with an error message instead of being applied. You are now thinking like a C++ programmer — that little check is exactly the kind of protection encapsulation is designed to make easy. See you in Lab 2.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST